Clase #03 de 27
UTF-8 & Strings

Abril 21, Martes

Agenda para esta clase

- Caracteres: Parte II
  - Ejemplo toupper
  - Abstracciones de Unicode
  - Bytes en Archivos
- UTF-8
- Introducción a Cadenas y Especificación de Length

Caracteres: Parte II

Ejercicio: To Upper (A Mayúsculas)

- Desarrollar la función **toupper**, *con precondiciones:* **char toupper(char);**
- Probar con **assert**
- ¿Es esta la versión estándar? ¿Es realmente una operación sobre carácter?

```
28  static void Test(PunteroAFuncion toupper){
29      assert('A'==toupper('a'));
30      assert('Z'==toupper('z'));
31      //assert('@'==toupper('@')); // No cumple precondición
32  }
33
34  char toupper0(char c){ return c - ('a'-'A');   }
35
36  char toupper1(char c){ return c - 'a' + 'A';   }
37
38  char toupper2(char c){ return c - 32;          }
39
40  char toupper3(char c){ return c - 0x20;        }
41
42  char toupper4(char c){ return c & 0xDF;        }
43
44  char toupper5(char c){ return c & 0b1101'1111; }
```

Niveles de Abstracción de Caracteres

Nivel	Nombre	Definición	Letra Latina A	Símbolo $	Letra Castellana Ñ	Emoji
1	**Glifo**	Representación pictórica de caracteres	A 𝒜 𝒶	$ ⧵$ 𝓢	Ñ 𝒩̃	👍
2	**Unidad de código**	Unidad de almacenamiento de Punto de código, parcial o completo.	01000001_2 141_{16} 65_{10}	00100100_2 24_{16} 36_{10}	$11000011\ 10010001_2$ $C3_{16}\ 91_{16}$ $195_{10}\ 145_{10}$	$11110000\ 10011111\ 10010001\ 10001101_2$ $F0_{16}\ 9F_{16}\ 91_{16}\ 8D_{16}$ $240_{10}\ 159_{10}\ 145_{10}\ 141_{10}$
3	**Puntos de código**	Valor numérico asignado un significado, en este caso mapeo entre valor y carácter.	U+0041	U+0024	U+004E + U+0303	U+1F44D + U+1F3FD
4	**Grupo de Grafemas (Cluster)**	Juntos generan un carácter	A	$	N + ~	👍 + 🟫
5	**Grafema**	Percibido por usuario, hace la diferencia entre palabras: Halo=HALO, HALO≠HOLA	A	$	Ñ	👍🏽
6	**Carácter ó Símbolo**	Concepto abstracto que representa el componente más chico del leguaje escrito y que tiene valor semántico.	A	$	Ñ	👍🏽
7	**Cadenas y Texto**	Secuencia de símbolos (de puntos de valor)	¡Hola 🌍!			

Visualizaciones: ASCII, Decimal, Hexadecimal

```
$ cat StreamsFilesBytes.cpp
#include <iostream>

int main() {
    std::cout << "ABCDEFG\n0123456\n !\"#$%\a\n";
}
$ c++ StreamsFilesBytes.cpp
$ ./a.out
ABCDEFG
0123456
 !"#$%
$ ./a.out > output.txt
$ cat output.txt
ABCDEFG
0123456
 !"#$%
$ hexdump -v -e '8/1 "%3_c "' -e '"\n"' output.txt
  A   B   C   D   E   F   G  \n
  0   1   2   3   4   5   6  \n
      !   "   #   $   %  \a  \n
$ hexdump -v -e '8/1 "%03u "' -e '"\n"' output.txt
065 066 067 068 069 070 071 010
048 049 050 051 052 053 054 010
032 033 034 035 036 037 007 010
$ hexdump -v -e '8/1 "%02x "' -e '"\n"' output.txt
41 42 43 44 45 46 47 0a
30 31 32 33 34 35 36 0a
20 21 22 23 24 25 07 0a
$ hexdump -C output.txt
00000000  41 42 43 44 45 46 47 0a  30 31 32 33 34 35 36 0a  |ABCDEFG.0123456.|
00000010  20 21 22 23 24 25 07 0a                           | !"#$%..|
```

Propósito	Unix	Windows
Representación carácter	`cat` *filename*	`type` *filename*
Redirección	*comando* > *destino*	
Representación numérica	`hexdump -C` *filename*	`debug` *filename* `certutil -encodehex` *filename* `stdout`

UTF-8

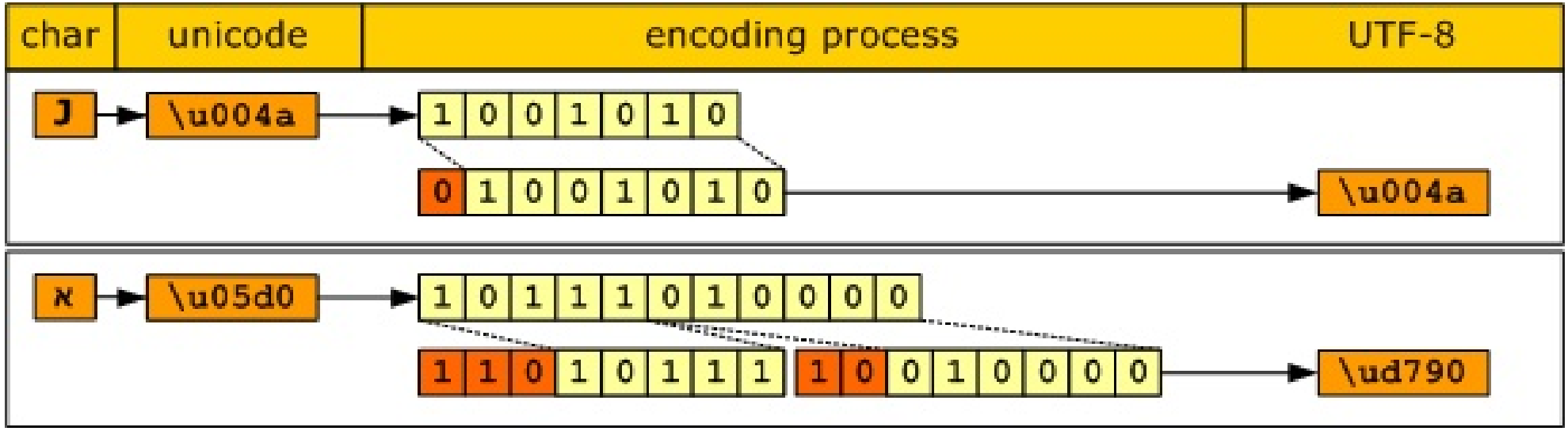
Unicode Transformation Format – 8-bit

Problema: Guardar Unicode Code Points

- Solución trivial: **Ancho fijo de 4 bytes**
  - Fácil, necesita 21 bits, entonces necesitamos **$2^1/_8$ bytes** (!?)…
  - mejor **3 bytes**…
  - pero es más eficiente mover de 32 bits, es decir **4 bytes**…
  - Ventajas: Discutir
  - Desventajas
    - 4 bytes para todos los caracteres, cuadruplica el tamaño
    - Muchos bytes en cero, que tienen significado especial
- Solución no trivial: **Ancho variable**
  - Codificación de Longitud variable, entre 1 y 6 bytes.
  - Si el code point es menor a 128, debe usar un solo byte
  - Con 4 bytes es suficiente para Unicode.
  - Ventajas:
    - No se extiende para el caso más común.
    - Compatible con ASCII.
  - Desventajas: Discutir
  - El 99% de las páginas transmiten en UTF-8

```
U-00000000 – U-0000007F: 0xxxxxxx
U-00000080 – U-000007FF: 110xxxxx 10xxxxxx
U-00000800 – U-0000FFFF: 1110xxxx 10xxxxxx 10xxxxxx
U-00010000 – U-001FFFFF: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
U-00200000 – U-03FFFFFF: 111110xx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx
U-04000000 – U-7FFFFFFF: 1111110x 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx
```

UTF-8: Ejemplos



UTF-8: Ejercicios

- Unicode: A U+0041 = 0100 0001_2 = 65_{10}
  - UTF-8: 0100 0001 = 0x41

- Unicode: © U+00A9 = 1010 1001_2 = 169_{10}
  - UTF-8: 1100 0010 1010 1001 = 0xC2 0xA9

- Unicode: ≠ U+2260 = 0010 0010 0110 0000_2 = 8.800_{10}
  - UTF-8: 11100010 10001001 10100000 = 0xE2 0x89 0xA0

Introducción a Strings

[MUCH2012] 1.1.2

String ó Cadena

- Secuencia **finita**
  - $\Sigma=\{a, b\}$
  - aab
- Pares ordenados, generado por **productos cartesianos**
  - ((a, a), b)
  - $\Sigma\times\Sigma\times\Sigma = (\Sigma\times\Sigma)\times\Sigma$
  - $\Sigma^2\times\Sigma$
  - Σ^3
- Aplanar: N-Tupla
  - (a, a, b)
- Sin paréntesis ni comas:
  - aab
  - $aab \in \Sigma^3$
  - $bab \in \Sigma^3$
  - $ab \notin \Sigma^3$
- Ejercicio 3:
  Dado el $\Sigma=\{if, for\}$, construya una cadena que tenga cuatro caracteres
- En C: `"aab"`

Cadenas Especiales

- Cadena vacía
  - ()
  -
  - Símbolos
    - Épsilon: ε
      ó
    - Lambda: λ
  - ¿A qué alfabeto pertenece?
    - (Evitar la respuesta obvia)
  - Discutir, ¿pertenece a todos los alfabetos o a ninguno?
  - En C: ""
- Cadena de un solo carácter
  - a (carácter)
    - En C: 'a'
  - a (string)
    - En C: "a"

Operaciones Sobre Strings

- Primitiva: Concatenación
- Derivadas
  1. Longitud
  2. Potenciación
  3. Proyección o Subindicación
  4. Reversa o Inversa
  5. Instancias de un carácter
  6. Contiene tal carácter
  7. Reemplazo u Sustitución
  8. Remoción
- Ejercicio, para cada operación ejemplo, definir conjunto de llega y de salida, por ejemplo:
  - Length : $String \rightarrow \mathbb{N}$
- ¿Cuáles operaciones son cerradas?

Operación: Length

- Otros Nombres
  - Length
  - GetLength
  - Longitud
  - Módulo
- Ejemplos
  - |aab|=3
  - length(aab)=3

- Especificación como función
  - Función
    - Relación entre conjuntos
      - *String*→ℕ
    - Fórmula
      - Recursividad, Caso base
      - Función partida
- Implementación: Próxima clase

$$\text{length}: \overset{\text{String}}{\Sigma^*} \longrightarrow \mathbb{N} / \text{length}(s) = \begin{cases} 0 & s = \varepsilon \\ 1 + \text{length}(t) & h \cdot \quad t,\ h \in \Sigma \end{cases}$$

Preguntas de Repaso

1. ¿Qué es un archivo? ¿Existen los tipos de archivos?
2. ¿Por qué se creó UTF-8?
3. ¿Qué relación hay entre UTF-8 y ASCII?
4. ¿Qué es una string?
5. ¿Cuál es la operación primitiva de strings? ¿Y alguna derivada?

Términos de la clase

Definir cada término con la bibliografía

- toupper
- precondiciones
- assert
- Archivo
- Formatos de visualización de archivos
- UTF-8
- Unidad de código
- Cadena ó String
- Operaciones de cadena
- Cadena vacía
- Cadena de un carácter
- Longitud de una cadena
- Especificación de operación como función

Referencias Bibliográficas

- Unicode: ISO/IEC 10646
- K&R
  - 7.8.2 Character Class Testing and Conversion
  - B.2 Character Class Tests: <ctype.h>
- MUCH: 1.2 Cadenas

Tareas para la próxima clase

1. Especificar e Implementar: Length
2. Leer Lenguaje Universal [MUCH]

¿Consultas?

Fin de la clase